Haskell Notes

目次

1 凡例 4
2 Haskell について 5
2.1 Haskell という森 5
2.2 関数型プログラミング 6
2.3 Haskell コンパイラの準備 7
2.4 余談：本書の構成 7
2.5 この章のまとめ 7
3 関数型という考え方 7
3.1 単語を数える 7
3.2 コマンドを関数として捉える 8
3.3 参照透過性と入出力 8
3.4 余談：参照透過性を強くサポートするプログラミング言語 9
3.5 この章のまとめ 9
4 カリー風な書き方 9
4.1 関数 9
4.2 ラムダ式 10
4.3 パタンマッチ・ガード・条件分岐 10
4.4 余談：局所変数 11
4.5 この章のまとめ 12
5 さらにカリー風な書き方 12
5.1 演算子 12
5.2 関数合成と関数適用 13
5.3 高階関数 14
5.4 余談：演算子の定義 14
5.5 この章のまとめ 15
6 型 15
6.1 データ型 15
6.2 カリー化 16
6.3 多相型と型クラス 18
6.4 余談：モノイド 18
6.5 この章のまとめ 18
7 リスト 18
7.1 リスト 18
7.2 畳み込み 18
7.3 マップ 18
7.4 余談：リストの実装 18
7.5 この章のまとめ 18
8 再帰 18
8.1 関数の再帰適用 18
8.2 末尾再帰 18
8.3 遅延評価 18
8.4 余談：クロージャ 18
8.5 余談：リストの実装 18

9 Maybe 18
9.1 Possibly 18
9.2 Maybe 18
9.3 リストと Maybe 18
9.4 余談: Either 18
9.5 この章のまとめ 18
10 RENEW 18
11 変数・関数・型 18
11.1 変数 18
11.2 変数の型 19
11.3 関数 19
11.4 有名な関数 20
11.5 関数の型 20
11.6 関数合成 20
11.7 IO サバイバルキット 1 20
12 条件分岐と再帰呼び出し 21
12.1 ラムダ 21
12.2 ローカル変数 21
12.3 クロージャ 22
12.4 型 22
12.5 条件 23
12.6 関数の再帰呼び出し 24
12.7 タプル 24
13 リスト 24
13.1 リスト 24
13.2 内包表記 25
13.3 文字列 25
13.4 マップと畳み込み 26
13.5 IO サバイバルキット 2 26
14 関手とモナド 27
14.1 Maybe 27
14.2 Maybe に対する計算 28
14.3 Maybe の中のリスト 28
14.4 型パラメタと型クラス 28
14.5 関手 29
14.6 関手としての関数 29
15 アプリカティブ関手 30
15.1 アプリカティブ関手 30
15.2 モナド 31
15.3 種 31
15.4 Data 31
15.5 型クラスとインスタンス 31
15.6 IO モナド 31
15.7 Do 構文 31
15.8 モノイド則 31
15.9 関手則 31
15.10 アプリカティブ関手則 31

15.11 モナド則 31
15.12 クラスの定義 32
16 Test Part 33
16.1 Function application 33
16.2 List map 33
16.3 Maybe map 33
16.4 Functor map 33
16.5 Applicative map 33
16.6 Apllicative map 2 33
16.7 Monadic function application 34
16.8 Bind 34
16.9 Double bind 34
16.10 Contextual map 34
17 Test Part 2 34
17.1 Lambda 34
17.2 Operators 34
17.3 Types 35

1 凡例

種類	字体・表記法	例
変数・関数	イタリック（1 文字）	x, f
有名な変数・関数・定数	ローマン・大文字	$\mathrm{first}, \mathrm{id}, \mathrm{otherwise}$
リスト変数	変数名に s をつける	x_{s}
Maybe 変数	変数名に ? をつける	$x_?$
一般のコンテナ変数	変数名に $*$ をつける	x_*
定数値コンストラクタ	ローマン・大文字	$\mathrm{True}, \mathrm{Nothing}$
値コンストラクタ	ローマン・大文字	Just_x
有名な定数値コンストラクタ	数学記号	$\emptyset, \oslash$
有名な値コンストラクタ	特別な括弧で包む	$[x], (y)$
アクション	ギリシア文字（1 文字）	α, μ
有名なアクション	サンセリフ	$\mathsf{main}, \mathsf{print}$
型（引数なし）	ボールドイタリック（1 文字）	$\boldsymbol{a}$
型（引数あり）	ボールドイタリック（1 文字）	$\boldsymbol{m_a}$
有名な型（引数なし）	ボールド・大文字	$\mathbf{Int}$
有名な型（引数あり）	特別な括弧で包む	$[\boldsymbol{a}], [\mathbf{Int}]$
型クラス	フラクチュール	$\mathfrak{Num}$
キーワード	固定幅	`let`
無名パラメタ	ひし形（白）	$\lozenge$
無名型パラメタ	ひし形（黒）	$\blacklozenge$
集合（数学）	ブラックボード（1 文字）	$\mathbb{Z}$
関手（数学）	スクリプト（1 文字）	$\mathscr{F}$
圏（数学）	カリグラフィ（1 文字）	$\mathcal{C}$

2 Haskell について

本書はプログラミング言語 Haskell の入門書である．それと同時に，本書はプログラミング言語を用いた代数構造の入門書でもある．プログラミングと代数構造の間には密接な関係があるが，とくに**関数型プログラミング**を実践する時にはその関係を意識する必要が出てくる．本書はその両者を同時に解説することを試みる．

2.1 Haskell という森

これからのプログラマにとって Haskell を無視することはできない．Haskell の「欠点をあげつらうことも，攻撃することもできるが，無視することだけはできない」のだ．それは Haskell がプログラミングの本質に深く関わっているからである．

Haskell というプログラミング言語を知ろうとすると，従来のプログラミング言語の知識が邪魔をする．モダンで，人気があって，Haskell から影響を受けた言語，たとえば Ruby や Swift の知識さえ，Haskell を学ぶ障害になり得る．ではどのようにして Haskell の深みに到達すればいいのだろうか．

その答えは，一見遠回りに見えるが，一度抽象数学の高みに登ることである．

と言っても，あわてる必要はない．

近代的なプログラミング言語を知っていれば，すでにある程度抽象数学に足を踏み入れているからである．そこで，本書では近代的なプログラマを対象に，プログラミング言語を登山口に抽象数学の山を登り，その高みから Haskell という森を見下ろすことにする．

さて，登山口にどのプログラミング言語を選ぶのが適当であろうか．TIOBE Index 2023 年 2 月版によると「ビッグ 5」として Python，C，C++，Java，C# が挙げられている．[1]順位の変動はあるが，他の調査でもビッグ 5 は過去何年も変動していないので，当座は妥当な統計であろう．このうち C は「多くのプログラマが読める」以外にメリットが無く，その唯一のメリットさえ最近は怪しくなっているため，登山口候補から外す．残るは Java，C# グループと Python ということになるが，シンプルであり，かつ Haskell と対極にある言語である Python を登山口に選ぶことにした．

本書では Python コードはこのように登場する．

```
print("Hello, world.")
```

本書に示すコードは擬似コードではなく，すべて実行可能な本物のコードである．

一部の章でどうしても型に触れないといけない部分がある．Python は動的型付け言語であり，型の説明には不適切であるため，この部分だけ理解の助けとして C++によるコードを例示した．この部分はコードを読まなくても先に進める．

ところで，プログラムのソースコードは現代でも ASCII 文字セットの範囲で書くことが標準的である．Unicode を利用したり，まして文字にカラーを指定したり，書体や装飾を指定することは一般的ではない．たとえば変数 a のことを a と書いたり $\boldsymbol{a}$ と書いたり $\tilde{a}$ と書いたりして区別することはない．

Haskell プログラマもまた，多くの異なる概念を同じ貧弱な文字セットで表現しなければならない．これは，はじめて Haskell コードを読むときに大きな問題になりえる．たとえば Haskell では `[a]` という表記をよく扱う．この `[a]` は `a` という変数 1 要素からなるリストのこともあるし，`a` 型という仮の型から作ったリスト型の場合もあるが，字面からでは判断できない．も

[1] https://www.tiobe.com/tiobe-index/

し変数はイタリック体，型はボールド体と決まっていれば，それぞれ $[a]$ および $[\boldsymbol{a}]$ と区別できたところである．

本書は，異なる性質のものには異なる書体を割り当てるようにしている．ただし，どの表現もいつでも Haskell に翻訳できるように配慮している．実際，本書執筆の最大の困難点は，数学的に妥当で，かつ Haskell の記法とも矛盾しない記法を見つけることであった．

2.2 関数型プログラミング

プログラマはなぜ Haskell を習得しなければならないのだろう．それは Haskell と**関数型プログラミング**の間に密接な関係があるからである．

関数型プログラミングとはプログラミングにおける一種のスローガンのようなもので，どの言語を用いたから関数型でどの言語を用いたから関数型ではない，というものではない．しかし，関数型プログラミングを強くサポートする言語と，そうでない言語とがある．ここら辺の事情はオブジェクト指向プログラミングとプログラミング言語の関係と似ている．Haskell は関数型プログラミングを強くサポートし，Python はほとんどサポートしない．

関数型プログラミングの特徴を一言で言えば，プログラム中の**破壊的代入**を禁止することである．変数 x に 1 という数値が一度代入されたら，変数 x の値をプログラム中に書き換える，すなわち破壊的代入をすることはできない．この結果，変数の値はプログラムのどこでも，どの時点で読み出しても同じであることが保証される．これを変数の**参照透過性**と呼ぶ．

プログラム全体に参照透過性があると，そのプログラムはブロックに分割しやすく，各々のブロックは再利用しやすい．またプログラムのどの断片から読み始めても，全体の構造を見失いにくい．これが関数型プログラミングとそれを強くサポートする Haskell を習得する理由である．

参照透過性がもたらすもう一つのボーナスは変数の**遅延評価**である．変数はいつ評価しても値が変わらないのだから，コンパイラは変数をできるだけ遅く評価してよい．この遅延評価によって，Haskell コンパイラは他の言語に見られない**無限リスト**を扱う能力を獲得している．

ここで数学とプログラミングの関係について述べておこう．ある方程式を解くためにコンピュータによって数値シミュレーションを行うとか，非常に複雑な微分を機械的に行うとか，プログラミングによって数学をサポートすることは計算機科学の主たる分野の一つであるが，ここではもっと根源的な話をする．

数学者もプログラマも**関数**をよく使う．数学者が使う関数とは，引数がいくつかあって，その結果決まる戻り値があるようなものだ．一方でプログラマが使う関数というのは，引数と戻り値はだいたい同じとして，中身に条件分岐があったり，ループがあったり，外部変数を書き換えたり，入出力をしたりする．

どちらも同じ関数であるのに，なぜこうもイメージが違うのだろうか．

もし，我々が関数型プログラミングの原則を忠実に守り，プログラム中のいかなる破壊的代入をも禁止するとすると，両者の関数は全く同じ性格になる．逐次実行も条件分岐もループも，それどころか定数さえ，**ラムダ式**という式だけで書けるようになる．あらゆるプログラムが，最終的には単一のラムダ式で書ける．

ところが，入出力，状態変数，例外など，プログラミングに使われる多くのテクニックは関数の副作用を前提としている．参照透過性と副作用を統一的に扱うためには**モナド**という数学概念が必要である．Haskell はモナドを陽に扱うプログラミング言語である．

2.3 Haskell コンパイラの準備

❇️ TK

2.4 余談：本書の構成

❇️ TK

2.5 この章のまとめ

❇️ TK

3 関数型という考え方

❇️ TK

3.1 単語を数える

関数型プログラミングという考え方自体は，プログラミングに必ずしも束縛されるものではない．

たとえばいま，ある英文テキストファイルから，単語を出現頻度順に取り出したいとしよう．こんなとき，プログラマならば次のように考えるのではないだろうか．

「まずは英文テキスト中の大文字を全部小文字にしよう」「それからアルファベットと空白以外を全部捨ててしまおう」「空白を改行に置き換えると，1 行に 1 単語になるな」「ここまで来たら出現頻度順に並べ直すのはわけないことだ」……

これらの言葉を UNIX の言葉に置き換えてみる．ここでは英文テキストを `the-great-gatsby.txt` としておこう．中身はフィッツジェラルドの名作「The Great Gatsby」（華麗なるギャッツビー）だ．最初の「まずは英文テキスト中の大文字を全部小文字にしよう」は UNIX コマンドではこうなる．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]'
```

この出力を適当なファイルにリダイレクトすれば，大文字がすべて小文字に置き換わったテキストファイルになる．

次にするのは「それからアルファベットと空白以外を全部捨ててしまおう」だ．UNIX コマンドではこうなる．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]'
```

これで単語だけのテキストファイルが出来る．次に空白をすべて改行に置き換えてしまおう．UNIX コマンドでこうする．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
```

1 行に 1 単語ずつ並べられたテキストファイルがこれで手に入ることになる．空行もあるが，それは問題ない．このファイルの中身をアルファベット順にソートする．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort
```

ソートできれば，同一の連続する行を数え上げて，一つの行にすればいい．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort | uniq -c
```

最後に，出現頻度順に逆順ソートをかけると「出現頻度順」になる．それには以下のようにする．

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort | uniq -c | sort -nr
```

この出力の先頭 10 行は，そのまま出現頻度上位 10 傑の単語となる．

3.2 コマンドを関数として捉える

さて，上述の出現頻度上位単語の抽出のどこが「関数型プログラミング」の考え方だったのだろうか．

実はこんな風に考えることが出来る．入力を x とする．いまの例では x がファイル `the-great-gatsby.txt` である．出力を y とする．ここに y は出現頻度順に並んだ出現頻度と単語のリストである．僕たちが欲しいのは

$$y = f(x) \tag{1}$$

となるようなプログラム f だった．プログラム f は簡単には手に入らないので，僕たちは UNIX コマンドの `tr` とか `sort` とか `uniq` とかをつなぎ合わせて作った．それらを「数学風」に書くと次のようになる．

$$\begin{aligned} f_1 &= \mathrm{tr}_{[A\ldots Z]\to[a\ldots z]} \\ f_2 &= \mathrm{tr}_{\overline{[a\ldots z,\square]}\to\emptyset} \\ f_3 &= \mathrm{tr}_{\square\to\swarrow} \\ f_4 &= \mathrm{sort} \\ f_5 &= \mathrm{uniq}_c \\ f_6 &= \mathrm{sort}_{n,r} \end{aligned} \tag{2}$$

そして x から y を得るために $y = f_6(f_5(f_4(f_3(f_2(f_1(x))))))$ という計算を行った．括弧が多すぎるので，この式を$y = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1(x)$ と書き直そう．ここに演算子 $\circ$ は「**関数合成演算子**」だ．

関数合成演算子を使うと，プログラム f を $f = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1$ と定義することも出来る．これが何を意味しているかと言うと，プログラムを小さな部分プログラムに分解したということだ．

分解したなら，合成する方法が必要になる．この例では，プログラムの各部分が他の部分に依存していないために，合成は数学的な関数合成と同じ方法が使える．

上記の例のように，数学における関数合成をそのままプログラミングに持ち込めれば，部分プログラムの合成も見通しが良いものになる．このような考え方が「関数型プログラミング」のエッセンスなのである．

3.3 参照透過性と入出力

TK

3.4 余談：参照透過性を強くサポートするプログラミング言語

TK

3.5 この章のまとめ

TK

4 カリー風な書き方

本書では一般の数学書やプログラミングの教科書からは少し異なった記法を用いる．ある概念が発明されてからずっと後になって正しい記法が見つかり，それがきっかけとなって正しく理解されるという現象は歴史上よくあることである．本書でも様々な新しい記号，記法を導入するが，この章では Haskell に近い記法から始めることにする．

4.1 関数

数学やプログラミミング言語には書き方に一定の決まりがある．この章ではまず「カリー風の」数式記述方式を見てみることにする．「カリー風」というのは，数学者ハスケル・カリーから名前を借りた言い方で，筆者が勝手に命名したものだ．

カリー風の書き方は数学の教科書やプログラミングの教科書で見かけるものとは若干違うが，圧倒的にシンプルで Haskell との親和性も高く，慣れてくると非常に読みやすいものなので，本書でも全面的に採用する．

まずは**関数**から見ていくことにしよう．Python や一般的な数学書では引数 x をとる関数 f を $f(x)$ と書くが，括弧は冗長なので今後は fx と書くことにする．[2]

関数 f に引数 xを「食わせる」ことを**関数適用**と呼ぶ．もし fx と書いてあったら，それは f と x の積，つまり $f \times x$ ではなく，従来の $f(x)$ すなわち関数 f に引数 x を与えているものと解釈する．高校生向けの数学書でも $\sin x$ のように三角関数に限ってはカリー風に書くことになっているので，まるで馴染みがないということもないだろう．なお，関数はいつも引数の左側に書くことにする．これを「関数 f が変数 x の左から作用する」と言い，また関数 f のことを**左作用素**とも呼ぶ．

複数引数をとる関数を Python や一般的な数学の教科書では $g(x, y)$ と書くが，これも括弧が冗長なので今後は gxy と書く．この場合式 gxy は左を優先して結合する．つまり $gxy = (gx)y$ である．これは引数 y に関数 (gx) が左から作用していると解釈する．関数 (gx) は引数 x に関数 g を作用させて作った関数である．引数に「飢えた」関数 (gx) を**部分適用**された関数と呼ぶ．

このように式の左側を優先的に演算していくことを**左結合**と呼ぶ．Haskell の場合，関数適用はいつも左結合である．

部分適用の例を見てみよう．例えばふたつの引数のうち大きい方を返す関数 max は $\max xy$ として使われるが，関数適用は左結合であるから $(\max x)y$ としても同じである．そこで $(\max x)$ だけ取り出すと，これは「引数が x よりも小さければ x を，そうでなければ引数を返す関数」とみなすことができる．[3]

---

[2] Haskell では関数 `f` に引数 `x` を適用させることを `f x` と書く．数学や物理学では x をパラメタとする関数を $f(x)$ と書く場合もあるし，f のようにパラメタを省略する場合もある．数学や物理学でパラメタを省略した場合は，$f(x_0)$ の意味で $f|_{x=x_0}$ と書くことがある．

[3] Haskell では $\max xy$ を `max x y` と書く．なお関数 $(\max x)$ のことを $\max_x$ と書く教科書も多い．関数引数を添え字で表す記法は，本書でも後に採用する．

4.2 ラムダ式

関数の正体は**ラムダ式**である．ラムダ式とは，仮の引数をとり，その値をもとになにがしかの演算を行い，その結果を返す式である．ラムダ式は名前のない関数のようなものだ．それゆえ，無名関数と呼ばれることもある．

例えば引数 x をとり値 $1+x$ を返すラムダ式を Python では

```
lambda x: 1 + x
```

と書くが，我々はより簡潔に $\backslash\, x \mapsto 1+x$ と書くことにする．

この式は多くの書物で $\lambda x\,.\,1+x$ と記述されるところである．しかし我々はすべてのギリシア文字を変数名のために予約しておきたいのと，ピリオド記号 . が今後登場する二項演算子と紛らわしいため，上述の記法を用いる．[4]

ラムダ式は関数である．ラムダ式を適用するには，ラムダ式を括弧で包む必要がある．例を挙げる．

$$(\backslash\, x \mapsto 1+x)2 \tag{3}$$

この式は結果として 3 を返す．

複数引数をとるラムダ式は例えば $\backslash\, xy \mapsto x+y$ のように引数を並べて書く．

本書では新たに，次のラムダ式記法も導入する．式中に記号 $\diamond$ が現れた場合，その式全体がラムダ式であるとみなす．記号 $\diamond$ の部分には引数が入る．第 n 番目の $\diamond$ には第 n 番目の引数が入る．例えばラムダ式 $\backslash\, xy \mapsto x+y$ は $\diamond+\diamond$ と書いても良い．式を左から読んで 1 番目の $\diamond$ が元々の x すなわち第 1 引数を，2 番目の $\diamond$ が元々の y すなわち第 2 引数を意味する．この省略記法はプログラミング言語 Scheme における `cut` プロシジャに由来する．[5]

4.3 パタンマッチ・ガード・条件分岐

関数の定義は，基本的にはラムダ式の変数への代入である．引数 x をとり値 $2x$ を返す関数 f は $f = \backslash\, x \mapsto 2\times x$ と定義できる．ただし，この省略形として $fx = 2\times x$ と書いても良い．[6]

関数に**スペシャルバージョン**がある場合はそれらを列挙する．例えば引数が 0 の場合は特別に戻り値が 1 であり，その他の場合は関数 f と同じ振る舞いをする関数 g を考える．このとき g は以下のように定義できる．これを関数の**パタンマッチ**と呼ぶ．[7]

$$\begin{aligned} g0 &= 1 \\ gx &= 2\times x \end{aligned} \tag{4}$$

[4] Haskell ではラムダ式 $\backslash\, x \mapsto 1+x$ を `\x -> 1 + x` と書く．ラムダ式は元々は $\hat{x}\,.\,1+x$ のように書かれていた．これが次第に $\hat{}x\,.\,1+x$ となり，$\Lambda x\,.\,1+x$ そして $\lambda x\,.\,1+x$ に変化していったと言われている．Haskell が λ の代わりに $\backslash$ 記号を使うのは，その形が似ているからである．

[5] Haskell では，中置演算子に限ってこの表現が使える．例えば $(\diamond+\diamond)$ は単に `(+)` と表現できる．ただし Scheme における `cut` プロシジャの `<>` は Haskell にはないため，Scheme でいう `(cut f <> y)` に相当するコードを直接は書けない．

[6] Haskell では $f = \backslash\, x \mapsto 2\times x$ を `f = \ x -> 2 * x` と書き，一方 $fx = 2\times x$ を `f x = 2 * x` と書く．

[7] Haskell では

```
g 0 = 1
g x = 2 * x
```

と書く．

関数のパタンマッチは，関数の内部に書いても良い．関数内部にパタンマッチを書きたい場合は次のように書く．

$$gx = \begin{cases} 0 \longrightarrow 1 \\ \sim\sim \longrightarrow \sin x/x \end{cases} \tag{5}$$

ここに $\sim\sim$ は任意の値の意味である．パタンマッチは上から順番にマッチングしていくため，この場合は 0 以外を意味する．[8]

一部のプログラミング言語では**デフォルト引数**という，引数を省略できるメカニズムがあるが，我々は引数をいつも省略しないことにする．[9]

関数定義にパタンマッチではなく**場合分け**が必要な場合は**ガード**を用いる．例えば引数の値が負の場合は 0 を，0 の場合は 1 を，それ以外の場合は関数 f と同じ振る舞いをする関数 h は以下のように定義する．[10]

$$\begin{aligned} hx|_{x<0} &= 0 \\ |_{x\equiv 0} &= 1 \\ |_{\text{otherwise}} &= fx \end{aligned} \tag{6}$$

関数定義の場合分けを駆使すれば**条件式**はなくても構わないが，条件式の記法があるのは便利である．Python には

```python
def f(x):
  if x == 0:
    return 1
  else:
    return sin(x)/x
```

のような**制御構造**としての条件文があるが，我々は値を持つ**条件式**を考える．

我々の条件式とは $fx = \texttt{if}\ x \equiv 0\ \texttt{then}\ 1\ \texttt{else}\ \frac{\sin x}{x}$ のように `if` 節，`then` 節，及び `else` 節からなるものであって，`then` 節も `else` 節も省略できないものとする．`if` 節の式の値が真 True であれば `then` 節の式が評価され，偽 False であれば `else` 節の式が評価される．我々の条件式は C における条件演算子（三項演算子）と等しく見えるが，Haskell の場合は遅延評価が行われ るため，結果として条件式の**短絡評価**が行われる点が異なる．[11]

4.4 余談：局所変数

TK

---

[8] Hhaskell では

```
g x = case x of 0 -> 1
                _ -> (sin x)/x
```

または `g x = case x of {0 -> 1; _ -> (sin x)/x}` と書く．

[9] Haskell にもデフォルト引数はない．

[10] Haskell では

```
h x | x < 0     = 0
    | x == 0    = 1
    | otherwise = f x
```

と書く．

[11] Haskell ではを $fx = \texttt{if}\ x \equiv 0\ \texttt{then}\ 0\ \texttt{else}\ \frac{\sin x}{x}$ を `f x = if x==0 then 1 else (sin x)/x` と書く．

4.5 この章のまとめ

TK

5 さらにカリー風な書き方

我々は関数とラムダ式の「カリー風」な書き方を見てきた．この章ではさらに演算子，関数合成についても「カリー風」な書き方を見ていく．

5.1 演算子

演算子は関数の特別な姿である．演算子は**作用素**と呼んでも良い．どちらも英語の operator の和訳である．演算子は普通アルファベット以外のシンボル 1 個で表現し，変数や関数の前に置いて直後の変数や関数に作用させるか，2 個の変数や関数の中間に置いてその両者に作用させる．例えば $-x$ のマイナス記号 $(-)$ は変数の前に置いて直後の変数 x に作用する演算子であり，$x+y$ のプラス記号 $(+)$ は 2 個の変数の間に置いてその両者 (x, y) に作用する．

1 個の変数または関数に作用する演算子を**単項演算子**と呼び，2 個の変数または関数に作用する演算子を**二項演算子**と呼ぶ．本書では単項演算子はすべて変数の前に置く，すなわち**前置**する．前置する演算子のことを**前置演算子**と呼ぶが，数学者は同じものを左作用素と呼ぶ．

Haskell には単項マイナス $(-)$ を除いて他に単項演算子はない．今後，誤解を避けるために数式中の $-x$ はすべて $(-x)$ と書く．

二項演算子のうちよく使われるものは和 $(+)$，積 $(\times)$，論理和 $(\vee)$，論理積 $(\wedge)$，同値 $(\equiv)$，大なり $(>)$，小なり $(<)$ 等である．二項演算子はたとえ積記号であっても省略できない．二項演算子は多数あるので，その都度説明する．[12]

二項演算子は**中置**することが基本であるが，括弧で包むことで前置することも可能である．任意の二項演算子 $\circ$ について $x \circ y$ 及び $(\circ)xy$ は全く同じ意味である．すなわち $(\circ)xy = x \circ y$ である．従って，二項演算子と 2 引数関数に本質的な差はない．本書では演算子と関数という用語は全く同じ意味で用いる．[13]

一般の関数が左結合であることを思い出すと，二項演算子を関数に見立てた $(\circ)$ も $(\circ)xy = ((\circ)x)y$ であるから，部分適用が可能である．この式から第 2 引数 y を取り除いて $(\circ)x$ という「餓えた」1 引数関数を取り出せる．例えば関数 $((+)1)$ は引数に 1 を加える関数である．[14]

前置される二項演算子 $(\circ)$ は，ラムダ式 $(\diamond \circ \diamond)$ の無名パラメタ $\diamond$ を省略したものと考えても良い．また $(\diamond \circ x)$ や $(x \circ \diamond)$ から無名パラメタを省略した $(\circ x)$ と $(x\circ)$ も有効な表現であり，特別に**セクション**と呼ばれる．

二項演算子 $\circ$ に対して $(\circ x)$ および $(x\circ)$ はそれぞれ以下の通りである．

$$
\begin{aligned}
(\circ x) &= (\diamond \circ x) \\
(x\circ) &= (x \circ \diamond) = (\circ)x
\end{aligned}
\tag{7}
$$

例えば $(1+)$ は $((+)1)$ と等価であり，これは $(+1)$ とも等価である．ただし，マイナス演算子 $(-)$ だけは例外で，(-1) はマイナス 1 を表す．負の数をいつも括弧で包んでおくのは良い

[12] Haskell では $\wedge$ を `&&` と書き，$\vee$ を `||` と書く．

[13] Haskell では任意の二項演算子を括弧で包むことで前置演算子として使うことができる．例えば `x + y` と `(+) x y` は同じ結果を返す．

[14] Haskell では $((+)1)$ を `((+)1)` と書く．

アイディアである．[15]

なお，二項演算子の結合性，すなわち左結合か右結合かは，演算子によって異なる．また演算の優先順位を明示的に与えるために括弧が用いられる．

一般の関数 f を中置演算子に変換する記号 $\langle f \rangle$ を今後用いる．この記号を用いると値 fxy のことを $x \langle f \rangle y$ と書くことができる．

5.2 関数合成と関数適用

ある変数に複数の関数を順に適用することはよくあることである．例えば

```
y = f(x)
z = g(y)
```

あるいは同じことであるが

```
z = g(f(x))
```

とすることがある．本書の記法で書けば $z = g(fx)$ である．この式から括弧を省略して $z = gfx$ としてしまうと，関数適用は左結合するから $z = (gf)x$ の意味になってしまう．関数 g が引数に関数をるので無い限り (gf) は無意味なので，$z = g(fx)$ の括弧は省略できない．

ここで，引数のことは忘れて，関数 f と関数 g を先に**合成**しておきたいとしよう．その合成を $g \circ f$ と書く．演算子 $\circ$ は**関数合成演算子**と呼ぶ．合成はラムダ式を使って $g \circ f = g(f \lozenge)$ と定義できる．関数合成演算子 $\circ$ は関数適用よりも優先順位が高く，$(g \circ f)x$ は単に $g \circ fx$ と書いても良い．この記法は括弧の数を減らすためにしばしば用いられる．[16]

関数合成演算子は，連続して用いることができる．関数合成演算子は左結合するので，関数 f, g, h について $h \circ g \circ f = (h \circ g) \circ f$ であるが，これを展開すると以下のようになる．

$$\begin{aligned}(h \circ g) \circ f &= (h \circ g)(f \lozenge) \\ &= h(g(f \lozenge)) \\ &= h \circ (g \circ f)\end{aligned} \tag{8}$$

そのため $h \circ g \circ f = h \circ (g \circ f)$ である．つまり，関数合成は順序に依存しない．

関数合成演算子とは逆に，結合の優先順位の低い**関数適用演算子**も考えておくと便利なこともある．関数適用演算子 $\$$ を次のように定義しておく．

$$f \,\$\, x = fx \tag{9}$$

演算子 $\$$ の優先順位は関数適用も含めあらゆる演算子よりも低いものとする．関数適用演算子を用いて $z = g(fx)$ を書き直すと $z = g \,\$\, fx$ となる．演算子 $\$$ の優先順位は足し算よりも低いので $f(x+1)$ は $f \,\$\, x + 1$ と書くこともできる．演算子 $\$$ を閉じ括弧のいらない開き括弧と考えてもよい．[17]

---

[15] Haskell は $(1+)$ を `(1+)` と書く．また `(-1)` はセクションではなくマイナス 1 を表す（`-1` というリテラルとみなされる）．ただし `(- 1)}` のように空白を挟んでも同じくマイナス 1 とみなされる（`1` というリテラルに単項マイナス演算子が適用される）．

[16] Haskell では関数 `g` と関数 `f` の合成は `g.f` である．式 $z = g \circ fx$ は `z = g.f x` と書く．

[17] Haskell では $g \,\$\, fx$ を `g $ f x` と書く．

関数適用演算子のもう一つの興味深い使い方は，関数適用演算子の部分適用である．セクション $(\diamond\,\$\,x)$ を用いると $(\diamond\,\$\,x)f = f\,\$\,x$ であるから，関数適用演算子を用いて引数を関数に渡すことができる．[18]

5.3 高階関数

関数を引数に取ったり，あるいは関数を返す関数のことを**高階関数**と呼ぶことがある．関数合成演算子と関数適用演算子は高階関数の好例である．

他に例えば，引数として整数 a を取り，関数 $fx = a + x$ を返すような関数 g を次のように定義することが出来る．

$$ga = a + \diamond \tag{10}$$

このとき，

$$\begin{aligned} f &= g100 \\ x &= f1 \end{aligned} \tag{11}$$

とすれば $x = 101$ を得る．[19]

高階関数は今後度々顔をだすことになる．後で登場するマップ演算子や畳込み演算子は高階関数の一種である．

5.4 余談：演算子の定義

Haskell では関数だけでなく，新しい演算子も定義できる．[20]

計算機科学者ドナルド・クヌースは，整数 x, n が与えられたとき x の n 乗を x^n ではなく $x \uparrow n$ と書いた．これは

$$x \uparrow n = \underbrace{x \times x \times \ldots \times x}_{n} \tag{12}$$

という意味である．[21]

クヌースはさらに演算子 $\uparrow\uparrow$ を

$$x \uparrow\uparrow n = \underbrace{x \uparrow x \uparrow \ldots \uparrow x}_{n} \tag{13}$$

のように定義した．これを**クヌースの矢印**と呼ぶ．クヌースの矢印は

$$\begin{aligned} x \uparrow\uparrow n \,|_{n \leq 0} &= 1 \\ |_{\text{otherwise}} &= x \uparrow (x \uparrow\uparrow (n-1)) \end{aligned} \tag{14}$$

と定義できる．[22]

---

[18] Haskell では $(\diamond\,\$\,x)f$ を `($x)f` と書く．

[19] Haskell では $ga = a + \diamond$ を $ga = \backslash x \mapsto a + x$ と展開しておいて `g a = \ x -> a + x` と書く．

[20] Haskell で演算子に使える記号は `! @ # $ % ^ & * - + = . \ | / < : > ? ~` の組み合わせである．

[21] Haskell では $x \uparrow n$ を `x ^ n` と書く．

[22] Haskell では次のように定義する．

```
x^^.n | n <= 0 = 1
      | otherwise = x^(x^^.(n-1))
```

なお厳密には

```
(^^.) :: Integral a => a -> a -> a
```

と演算子の型を宣言しておく必要がある．

なおこの定義は自分自身を呼び出す**再帰**を行っている．再帰に関しては〜〜〜で詳しく述べる．

5.5 この章のまとめ

TK

6 型

Haskell の変数，関数にはすべて型がある．プログラマの言う型とは，数学者の言う集合のことである．本章では，Haskell が扱う基本的な型であるデータ型と，パラメトリックな型である多相型，および型の型である型クラスについて述べる．また関数のカリー化についても述べる．

6.1 データ型

型とは変数が取りうる値に言語処理系が与えた制約のことである．Haskell を含む多くのコンパイラ言語は**静的型付け**と言って，コンパイル時までに変数の型が決まっていることをプログラマに要求する．一方，Python のようなインタプリタ言語はたいてい**動的型付け**と言って，プログラムの実行時まで変数の型を決めない．

変数に型の制約を設ける理由は，プログラム上のエラーが減ることを期待するためである．例えば真理値が必要とされるところに整数値の変数が来ることは悪い予兆である．一方で C 言語のように全ての変数にいちいち型を明記していくのも骨が折れる．

数学者や物理学者は変数に型の制約を求める一方，新しい変数の型は明記せず読者に推論させる方法をしばしばとる．例えば，質量 m は「スカラー」という型を持つし，速度 v は「3 次元ベクトル」という型を持つ．スカラーと 3 次元ベクトルの間に足し算は定義されていないため，例えば $m + v$ という表記を見たときに，両者の型を知っていれば直ちにエラーであることがわかる．

Haskell はコンパイラが型推論を行うことで，型が自明の場合は型を省略することができる．

Haskell にはよく使う型が予め用意されている．例えば**論理型**は論理値すなわち真 (True) または偽 (False) という値をとる変数の型である．ある変数 x が論理型であることを，Haskell では $x :: \mathbf{Bool}$ と書く．数学者なら同じことを $x \in \mathbb{B}$ と書くところであるが，ここは Haskell の流儀に従おう．また型定義と値定義はよく一緒に行われるので，今後 $x :: \mathbf{Bool} = \mathrm{True}$ のようにまとめて書くことにしよう．[23]

この場合変数 x が **Bool** 型であることは自明であるため $x :: \mathbf{Bool}$ は省略できるが，可能な限り型を明記しておくことは良い習慣である．また必要に応じて変数に型名を注釈することがある．例えば $x = \mathrm{True} :: \mathbf{Bool}$ のように書く場合がある．[24]

他に整数を表す**整数型**がある．整数型には 2 種類あって，その一つは **Int** である．この **Int** は C の `int` と似た「計算機にとって都合の良い整数」である．計算機にとって都合の良い整数とは，例えば 64 ビット計算機の場合 -2^{63} から $2^{63} - 1$ の間の整数という意味である．[25]

---

[23] Haskell では

```
x :: Bool
x = True
```

と書く．

[24] Haskell では $x = \mathrm{True} :: \mathbf{Bool}$ を `x = True :: Bool` と書く．

[25] Haskell では **Int** を `Int` で表す．

整数型には **Integer** もある．この **Integer** は計算機にとっては非常識なぐらい大きな，あるいは小さな値を表すことができる．[26]

計算機は残念ながら無限精度の実数を扱えない．そこで標準精度（単精度）の**浮動小数点数型**である **Float** と，**倍精度浮動小数点型**である **Double** が提供される．[27]

もう一つ，計算機ならではの型がある．それは **Int** とよく似ているが，特別に文字を扱うために考えられた**文字型** **Char** である．文字といってもその中身は整数である．整数ではあるが，わざわざ別な型とするのには理由がある．[28]

理由の第一は，文字が小さな整数であるため，文字型を独立して定義しておくことでメモリを節約できるのである．特にメモリが高価であった時代はこれが唯一の理由であった．現在でも，整数が一般に 64 ビットを消費するのに対し，UTF-8 文字エンコードを用いている場合，アルファベットは 8 ビットしか消費しない．

理由の第二は，単純に整数と文字が異なるからである．文字を表す変数に整数を代入するのは悪い兆しである．

理由の第三は，文字が数値にエンコードされる方式が可変長である場合に備えて，整数と区別しておくためである．例えば UTF-8 文字エンコードは可変長エンコーディングを行う．

このような基本的な型を**データ型**と呼ぶ．

関数にも型がある．例えば整数引数を一つ取り，整数を返す関数 f は $f :: \mathbf{Int} \to \mathbf{Int}$ という型を持つ．この型は

$$f :: \underbrace{\mathbf{Int}}_{x} \to \underbrace{\mathbf{Int}}_{fx} \tag{15}$$

のようにイメージすると良い．これは関数 f が集合 **Int** から集合 **Int** への**写像**であると読む．[29]

6.2 カリー化

Haskell では，どのような関数であれ引数は 1 個しかとらない．引数が 2 個あるように見える関数として，例えば gxy があったとしよう．ここに g は関数，x と y は変数である．関数適用は左結合であるから，これは $(gx)y$ である．ここに (gx) は引数 y をとる関数であると見ることができる．つまり，関数 g とは引数 x をとり「引数 y をとって値を返す関数 (gx) を返す」関数であると言える．

二項演算 $x + y$ は $(+)xy$ とも書けたことを思い出そう．これも左結合を思い出すと $(+)xy = ((+)x)y$ であるから，y という引数を $((+)x)$ という関数に食わせていると解釈できる．

ラムダ式の場合は話はもっと単純で，形式的に

$$\backslash\, xy \mapsto x + y = \backslash\, x \mapsto (\backslash\, y \mapsto x + y) \tag{16}$$

のように展開すれば 1 引数にできる．矢印 $\mapsto$ は**右結合**である．そこでこのラムダ式は括弧を省略して

$$\backslash\, xy \mapsto x + y = \backslash\, x \mapsto \backslash\, y \mapsto x + y \tag{17}$$

[26] Haskell では **Integer** を `Integer` で表す．
[27] Haskell では **Float**, **Double** をそれぞれ `Float` と `Double` で表す．
[28] Haskell では **Char** を `Char` で表す．
[29] Haskell では `f :: Int -> Int` と書く．

とも書かれる．

複数引数をとる関数を 1 引数関数に分解することを**カリー化**と呼ぶ．これはこの分野の先駆者であるハスケル・カリーの名前に由来する．

整数引数を二つ取り，整数を返す関数 g は

$$g :: \mathbf{Int} \to \mathbf{Int} \to \mathbf{Int} \tag{18}$$

という型を持つ．写像の矢印記号は右結合するので，これは

$$g :: \mathbf{Int} \to (\mathbf{Int} \to \mathbf{Int}) \tag{19}$$

と同じ意味である．上式は

$$g :: \underbrace{\mathbf{Int}}_{x} \to \underbrace{\overbrace{\mathbf{Int}}^{y} \to \overbrace{\mathbf{Int}}^{(gx)y}}_{gx} \tag{20}$$

のようにイメージすると良い．

自然言語で考えると **Int** 型の引数を一つ取り，**Int** 型の引数を一つ取って **Int** 型の値を返す関数を返す，と読める．[30]

Haskell には**タプル**という型がある．タプルとは，複数の変数を組み合わせたもので，例えば変数 x と y をひとまとめにした (x, y) はタプルである．変数 x, y の型は同じでも良いし，異なっても良い．[31]

[30] 関数の型に出てくる $\to$ は 2 引数をとる型コンストラクタである．型コンストラクタに関しては〜〜〜で詳しく述べる．例えば $\boldsymbol{a} \to \boldsymbol{b}$ という型は $(\to)\boldsymbol{ab}$ の別名であり，型コンストラクタ $(\to)$ に引数 $\boldsymbol{a}$ と $\boldsymbol{b}$ を与えたものと読む．

[31] Haskell では (x, y) を `(x, y)` と書く．

6.3 多相型と型クラス

6.4 余談：モノイド

6.5 この章のまとめ

7 リスト

7.1 リスト

7.2 畳み込み

7.3 マップ

7.4 余談：リストの実装

7.5 この章のまとめ

8 再帰

8.1 関数の再帰適用

8.2 末尾再帰

8.3 遅延評価

8.4 余談：クロージャ

8.5 余談：リストの実装

9 Maybe

9.1 Possibly

9.2 Maybe

9.3 リストと Maybe

9.4 余談: Either

9.5 この章のまとめ

10 RENEW

11 変数・関数・型

TK

11.1 変数

変数 x に値 1 を代入するには次のようにする．[32]

[32] Haskell では `x = 1` と書く．

$$x = 1 \tag{21}$$

変数という呼び名に反して，変数の値は一度代入したら変えられない．そこで変数に値を代入するとは呼ばずに，変数に値を**束縛**するという．式 21 の右辺のように数式にハードコードされた値を**リテラル**と呼ぶ．

リテラルや変数には**型**がある．型は数学者の**集合**と似た意味で，整数全体の集合 $\mathbb{Z}$ に相当する**整数型**や，実数全体の集合 $\mathbb{R}$ に相当する**浮動小数点型**がある．整数と整数型，実数と浮動小数点型は異なるため，整数型を **Int** で，浮動小数点型を **Double** で表すことにする．[33]

数学者は変数 x が整数であることを $x \in \mathbb{Z}$ と書くが，本書では $x :: \mathbf{Int}$ と書く．これは記号 $\in$ を別の用途に用いるためである．[34]

本書では変数名を原則 1 文字として，イタリック体で表し w, x, y, z のような n 以降のアルファベットを使う．

変数の値がいつでも変化しないことを**参照透過性**と呼ぶ．プログラマが変数の値を変化させたい，つまり**破壊的代入**を行いたい理由はユーザ入力，ループ，例外，内部状態の変化，大域ジャンプ，継続を扱いたいからであろう．しかし，後に見るようにループ，例外，内部状態の変化，大域ジャンプ，継続に変数の破壊的代入は必要ない．ユーザ入力に関しても章を改めて取り上げる．参照透過性を強くサポートするプログラミング言語をを**関数型プログラミング言語**と呼ぶ．

11.2 変数の型

TK

$$x :: \mathbf{Int} \tag{22}$$

$$x = 1 \tag{23}$$

$$x :: \mathbf{Int} = 1 \tag{24}$$

11.3 関数

整数 x に 1 を足す**関数** f は次のように定義できる．

$$fx = x + 1 \tag{25}$$

ここに x は関数 f の**引数**である．引数は括弧でくるまない．[35]

本書では関数名を原則 1 文字として，イタリック体で表し，f, g, h のようにアルファベットの f 以降の文字を使う．ただし有名な関数についてはローマン体で表し，文字数も 2 文字以上とする．たとえば sin などの三角関数や指数関数がそれにあたる．

変数 x に関数 f を**適用**する場合は次のように書く．ここでも引数を括弧でくるまない．[36]

$$z = fx \tag{26}$$

関数 f が引数をふたつ取る場合は，次のように書く．[37]

$$z = fxy \tag{27}$$

[33] Haskell では `Int` および `Double` と書く．
[34] Haskell では $x :: \mathbf{Int}$ を `x :: Int` と書く．
[35] Haskell では `f x = x + 1` と書く．
[36] Haskell では `z = f x` と書く．
[37] Haskell では `z = f x y` と書く．

なお fxy は $(fx)y$ と解釈される．前半の (fx) は 1 引数の関数とみなせる．2 引数関数を連続した 1 引数関数の適用とみなす考え方を，関数の**カリー化**と呼ぶ．

11.4 有名な関数

TK

11.5 関数の型

TK

11.6 関数合成

変数 x に関数 f と関数 g を連続して適用したい場合 $z = g(fx)$ とするところであるが，事前に関数 f と関数 g を**合成**しておきたいことがある．

関数の合成は次のように書く．

$$k = g \circ f \tag{28}$$

関数の連続適用 $g(fx)$ と合成関数の適用 $(g \circ f)x$ は同じ結果を返す．[38]

関数合成演算子 $\circ$ は以下のように**左結合**する．

$$\begin{aligned} k &= h \circ g \circ f \\ &= (h \circ g) \circ f \end{aligned} \tag{29}$$

関数適用のための特別な演算子 $ があると便利である．演算子 $ は関数合成演算子よりも優先順位が低い．例を挙げる．[39]

$$\begin{aligned} z &= h \circ (g \circ f)x \\ &= h \,\$\, g \circ fx \end{aligned} \tag{30}$$

11.7 IO サバイバルキット 1

プログラムとは合成された関数である．多くのプログラミング言語では，プログラムそのものに main という名前をつける．本書では「**IO モナド**」の章で述べる理由によって，main 関数をサンセリフ体で main と書く．

実用的なプログラムはユーザからの入力を受け取り，関数を適用し，ユーザへ出力する．Haskell ではユーザからの 1 行の入力を getLine で受け取り，変数の値を print で書き出せる．ここに getLine と print は関数（ファンクション）ではあるが，特別に「**アクション**」とも呼ぶ．関数 main もアクションである．

引数 x の 2 乗を求める関数 f は次のように定義できる．

$$\begin{aligned} f &:: \mathbf{Double} \to \mathbf{Double} \\ fx &= x \times x \end{aligned} \tag{31}$$

ユーザからの入力に関数 f を適用してユーザへ出力するプログラムを Haskell で書くと次のようになる．

$$\mathsf{main} = \mathsf{print} \circ f \circ \mathrm{read} \heartsuit \mathsf{getLine} \tag{32}$$

[38] Haskell では $k = g \circ f$ を `k = g . f` と書く．
[39] Haskell では $z = h \,\$\, g \circ fx$ を `z = h $ (g . f) x` と書く．

ここに関数 read は**文字列**であるユーザ入力を数に変換する関数である．また演算子 ♡ は新たな関数合成演算子で，アクションとアクションを合成するための特別な演算子である．詳細は「**モナド**」の章で述べる．

- Haskell では式 31 と式 32 をまとめて次のように書く．

```haskell
f :: Double -> Double
f x = x * x

main = print . f . read =<< getLine
```

12 条件分岐と再帰呼び出し

12.1 ラムダ

関数とは，変数名に束縛された**ラムダ式**である．引数をひとつとり，その引数に 1 を足して返す関数 f はラムダ式を用いて次のように書ける．

$$f = \backslash\, x \mapsto x + 1 \tag{33}$$

ラムダ式は入れ子に出来る．引数をふたつとり，その引数同士を足すラムダ式は次のように書ける．

$$\backslash\, x \mapsto (\backslash\, y \mapsto x + y) \tag{34}$$

より簡潔に式 34 を

$$\backslash\, xy \mapsto x + y \tag{35}$$

と書いても良い．

本書では無名変数 $\diamond$ を用いた以下の書き方も用いる．[40]

$$\begin{aligned} f &= (\diamond + 1) \\ &= \backslash\, x \mapsto x + 1 \end{aligned} \tag{36}$$

無名変数が 2 回以上登場した場合は，その都度新しいパラメタを生成する．たとえば次のとおりである．[41]

$$\begin{aligned} f &= \diamond + \diamond \\ &= \backslash\, x \mapsto \backslash\, y \mapsto x + y \\ &= \backslash\, xy \mapsto x + y \end{aligned} \tag{37}$$

12.2 ローカル変数

関数内で**ローカル変数**を使いたい場合は以下のように行う．[42]

$$z = \texttt{let}\ y \equiv 1\ \texttt{in}\ x + y \tag{38}$$

ローカル変数はラムダ式のシンタックスシュガーである．式 38 は次の式と等価である．

$$z = (\backslash\, y \mapsto x + y)1 \tag{39}$$

[40] 無名変数は Haskell には無いが，代わりに「セクション」という書き方ができる．式 $f = (\diamond + 1)$ を Haskell では `f = (+1)` と書く．

[41] Haskell では $f = (\diamond + \diamond)$ を `f = (+)` と書く．

[42] Haskell では $z = \texttt{let}\ y \equiv 1\ \texttt{in}\ x + y$ を `z = let y = 1 in x + y` と書く．

ローカル変数の定義は次のように後置できる．[43]

$$z = x + y \texttt{ where } y \equiv 1 \tag{40}$$

12.3 クロージャ

ラムダ式を返す関数は，ラムダ式内部に値を閉じ込めることができる．たとえば

$$fn = \backslash\, x \mapsto n + x \tag{41}$$

のように関数を定義して良い．関数 f に引数 n を与えると，新たな 1 引数関数が得られる．例を挙げる．

$$\begin{aligned} n &= 3 \\ g &= fn \end{aligned} \tag{42}$$

この例では，関数 g の中に値 $n = 3$ が閉じ込められているため $g1$ は 4 と評価される．値を閉じ込めたラムダ式を**クロージャ**と呼ぶ．

12.4 型

TK 一部前倒し

すべての変数，関数には**型**がある．代表的な型には整数型，浮動小数点型，ブール型，文字型がある．整数型を **Int** で，浮動小数点型を **Double** で表すことはすでに述べたとおりである．

Haskell には 2 種類の整数型がある．ひとつは**固定長整数型**で，もうひとつは**多倍長整数型**である．Haskell では前者を `Int` で，後者を `Integer` で表す．多倍長整数型はメモリの許す限り巨大な整数を扱えるので，整数全体の集合に近いのであるが，本書では主に固定長整数型を用いる．

浮動小数点型には**単精度浮動小数点型**と**倍精度浮動小数点型**があり，Haskell では前者を `Float` で，後者を `Double` で表現する．単精度浮動小数点型はめったに用いられないため，本書では浮動小数点型と言えば倍精度浮動小数点型を意味することにする．

論理型は論理値 True または False のいずれかしか値をとれない型である．論理型のことを **Bool**と書く．[44]

文字型を **Char** と書く．[45]

変数 x の型が **Int**のとき，以下のように**型注釈**を書く．[46]

$$x :: \mathbf{Int} \tag{43}$$

1 引数関数の型は次のように注釈できる．[47]

$$f :: \mathbf{Int} \to \mathbf{Int} \tag{44}$$

ここで関数 f は整数型の引数をひとつとり，整数型の値を返す．[48]

---

[43] Haskell では $z = x + y$ `where` $y \equiv 1$ を `z = x + y where y = 1` と書く．
[44] Haskell ではブール型を `Bool` と書く．
[45] Haskell では Unicode 文字型を `Char` と書く．
[46] Haskell では `x :: Int` と書く．
[47] Haskell では `f :: Int -> Int` と書く．
[48] 正確には `->` は型コンストラクタである．無名の型引数を $\blacklozenge$ で表すと `->` は $\langle \blacklozenge \to \blacklozenge \rangle$ という，ふたつの型引数を取る型コンストラクタである．

2 引数関数の方は次のように注釈できる．[49]

$$f :: \mathbf{Int} \to \mathbf{Int} \to \mathbf{Int} \tag{45}$$

ここで関数 f は整数型の引数をふたつとり，整数型の値を返す．型 $\mathbf{Int} \to \mathbf{Int} \to \mathbf{Int}$ は $\mathbf{Int} \to (\mathbf{Int} \to \mathbf{Int})$ と解釈される．

$(\mathbf{Int} \to \mathbf{Int})$ 型の関数を受け取り $(\mathbf{Int} \to \mathbf{Int})$ 型の関数を返す関数は次の型を持つ．[50]

$$f :: (\mathbf{Int} \to \mathbf{Int}) \to (\mathbf{Int} \to \mathbf{Int}) \tag{46}$$

なお後半の括弧は省略可能なので次のように書いても良い．

$$f :: (\mathbf{Int} \to \mathbf{Int}) \to \mathbf{Int} \to \mathbf{Int} \tag{47}$$

Haskell ではすべての変数, 関数に型があり, 型はコンパイル時に決定されていなければならない．ただし，式から**型推論**が行える場合は型注釈を省略できる．

12.5 条件

条件分岐は次のように書く．[51]

$$z = \texttt{if}\ x > 0\ \texttt{then}\ x\ \texttt{else}\ (-x) \tag{48}$$

$$f = \texttt{case}\ x\ \texttt{of} \begin{cases} 1 \longrightarrow 1 \\ \sim\sim \longrightarrow 0 \end{cases} \tag{49}$$

この場合 $x \equiv 1$ ならば f は 1 を，そうでなければ f は 0 を返す．ここに $\sim\sim$ はすべてのパターンに一致する記号である. パターンマッチは上から順に行われる. 条件分岐の代わりに以下のような**パターンマッチ**も使える．[52]

関数定義にもパターンマッチを使える．[53]

$$\begin{aligned} f1 &= 1 \\ f\sim\sim &= 0 \end{aligned} \tag{50}$$

関数定義には次のように**ガード**と呼ばれる条件を付与することができる．[54]

$$\begin{aligned} fx \mid_{x>0} &= x \\ \mid_{\text{otherwise}} &= (-x) \end{aligned} \tag{51}$$

ここに otherwise は True の別名である．

ガードは上から順にマッチされる．

---

[49] Haskell では `f :: Int -> Int -> Int` と書く．

[50] Haskell では `f :: (Int -> Int) -> (Int -> Int)` と書く．

[51] Haskell では `z = if x > 0 then x else (-x)` と書く．

[52] Haskell では以下のように書くのが一般的である．

```
f = case x of 1 -> 1
              _ -> 0
```

[53] Haskell では次のように書く．

```
f 1 = 1
f _ = 0
```

[54] Haskell では次のように書く．

```
f x | x > 0     = x
    | otherwise = (-x)
```

12.6 関数の再帰呼び出し

関数は再帰的に呼び出せる．$n \geq 0$ を前提とすると，n 番目のフィボナッチ数を計算する関数 fib を次のように定義できる．[55]

$$\begin{aligned} \mathrm{fib}\ 0 &= 0 \\ \mathrm{fib}\ 1 &= 1 \\ \mathrm{fib}\ n &= \mathrm{fib}\ (n-1) + \mathrm{fib}\ (n-2) \end{aligned} \tag{52}$$

12.7 タプル

複数の変数をまとめてひとつの**タプル**にすることができる．例を挙げる．[56]

$$z = (x, y) \tag{53}$$

タプルの型は，要素の型をタプルにしたものである．例えば **Int** が 2 個からなるタプルの型は次のようになる．[57]

$$z :: (\mathbf{Int}, \mathbf{Int}) \tag{54}$$

要素を含まないタプルを**ユニット**と呼ぶ．ユニットは次のように書く．[58]

$$z = () \tag{55}$$

ユニットの型は**ユニット型**で，型注釈を次のように書く．[59]

$$z :: () \tag{56}$$

13 リスト

13.1 リスト

任意の型について，その型の要素を並べた列を**リスト**と呼ぶ．

ある変数がリストであるとき，その変数がリストであることを忘れないように x_{s} と小さく s を付けることにする．

空リストは次のように定義する．[60]

$$x_{\mathrm{s}} = \emptyset \tag{57}$$

任意のリストは次のように**リスト構築演算子** : を用いて構成する．

$$x_{\mathrm{s}} = x_0 : x_1 : x_2 : \ldots : \emptyset \tag{58}$$

リストの型はその構成要素の型をブラケットで包んで表現する．[61]

---

[55] Haskell では次のように書く．ただし Haskell には符号なし整数型がないために n が正であることを別に担保する必要がある．またこのコードは無駄な再帰呼び出しを行っており実用的ではない．

```
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

[56] Haskell では `z = (x, y)` と書く．

[57] Haskell では `z :: (Int, Int)` と書く．

[58] Haskell では `z = ()` と書く．

[59] Haskell では `z :: ()` と書く．

[60] Haskell では `xs = []` と書く．

[61] Haskell では `xs :: [Int]` と書く．

$$x_{\mathrm{s}} :: [\mathbf{Int}] \tag{59}$$

リストは次のように構成することもできる．[62]

$$x_{\mathrm{s}} = [1, 2...100] \tag{60}$$

なお次のような**無限リスト**を構成しても良い．[63]

$$x_{\mathrm{s}} = [1, 2...] \tag{61}$$

リストとリストをつなぐ場合は**リスト結合演算子** ※ を用いる．[64]

$$z_{\mathrm{s}} = x_{\mathrm{s}} \mathbin{※} y_{\mathrm{s}} \tag{62}$$

関数はリストを受け取ることができる．次の書き方では，関数 f は整数リストの最初の要素 x と残りの要素 x_{s} を別々に受け取り，先頭要素だけを返す．[65]

$$\begin{aligned} f &:: [\mathbf{Int}] \to \mathbf{Int} \\ f(x : x_{\mathrm{s}}) &= x \end{aligned} \tag{63}$$

ただし，引数のリストが空リストである可能性を考慮して，式 63 は次のように書き直すべきである．

$$\begin{aligned} f &:: [\mathbf{Int}] \to \mathbf{Int} \\ f\emptyset &= 0 \\ f(x : x_{\mathrm{s}}) &= x \end{aligned} \tag{64}$$

$f\emptyset$ が 0 を返すのは不自然だが，関数fの戻り型を整数型としているためこれは仕方がない．エラーを考慮する場合は節 14.1 で述べる Maybe を使う必要がある．

リストのリストは次のように構成できる．[66]

$$x_{\mathrm{ss}} = [[1, 2], [3, 4]] \tag{65}$$

13.2 内包表記

リストの構成には**内包表記**が使える．例を挙げる．[67]

$$x_{\mathrm{s}} = [x^2 \mid x \in [1, 2...100], \mathrm{even}\ x] \tag{66}$$

関数 even は引数が偶数の場合にだけ True を返す関数である．この例では数列 $[1, 2...100]$ のうち偶数だけを 2 乗したリストを作っている．

13.3 文字列

文字型のリストを文字列型と呼び **String** で表す．**String** 型は次のように予約語 `type` を用いて，**型シノニム**すなわち型の別名として次のように定義される．

---

[62] Haskell では `xs = [1, 2..100]` と書く．ピリオドの数に注意しよう．

[63] Haskell では `xs = [1, 2..]` と書く．

[64] Haskell では `zs = xs ++ ys` と書く．

[65] この関数 f の実装は Haskell の `head` 関数と同じである．Haskell では `head` 関数の使用は非推奨となっており，代わりに `headMaybe` 関数が推奨されている．

[66] Haskell では `xss = [[1,2],[3,4]]` と書く．

[67] Haskell では次のように書く．

```
xs = [x^2 | x <- [1, 2..100], x > 50]
```

$$\texttt{type}\ \mathbf{String} = [\mathbf{Char}] \tag{67}$$

文字列型のリテラルは次のように書く．[68]

$$s = \texttt{"Hello, World!"} \tag{68}$$

リストに対するすべての演算は文字列にも適用可能である．

13.4 マップと畳み込み

リスト x_{s} の各要素に関数 f を適用して，その結果をリスト z_{s} に格納するためには次のようにマップ演算子 $*$ を用いる．[69]

$$z_{\mathrm{s}} = f * x_{\mathrm{s}} \tag{69}$$

式 69 は次の式と同じである．[70]

$$z_{\mathrm{s}} = [fx \mid x \in x_{\mathrm{s}}] \tag{70}$$

リスト x_{s} の各要素を先頭から順番に二項演算子を適用して，その結果を得るには畳み込み演算子を用いる．たとえば整数リストの和は次のように書ける．[71]

$$\begin{aligned} z &= \bigcup_{0}^{\lozenge+\lozenge} x_{\mathrm{s}} \\ &= a + x_0 + x_1 + \ldots + x_n \end{aligned} \tag{71}$$

Haskell では $\sum = \cup_0^{\lozenge+\lozenge}$ として関数 $\sum$ が定義されている．[72]

リスト x_{s} が $x_{\mathrm{s}} = [x_0, x_1, \ldots, x_n]$ のとき，一般に $\cup_a^{\lozenge\square\lozenge}\ x_{\mathrm{s}} = a \square x_0 \square x_1 \square \ldots \square x_n$ が成り立つ．ここに $\square$ は任意の二項演算子である．

畳み込み演算子には次の右結合バージョンが存在する．[73]

$$\begin{aligned} z &= \bigsqcup_{0}^{\lozenge\square\lozenge} x_{\mathrm{s}} \\ &= a \square (x_0 \square (x_1 \square \ldots \square (x_{n-2} \square (x_{n-1} \square x_n)) \ldots)) \end{aligned} \tag{72}$$

13.5 IO サバイバルキット 2

1 行ごとに 3 次元ベクトルが並べられた，以下の入力ファイルがあるとする．

```
1.0 2.0 3.0
4.5 5.5 6.5
```

このようなファイル形式は計算機科学者にとって見慣れたものである．

各行つまり各ベクトルごとに，そのノルムを計算して出力するプログラムを書きたいとしよう．まず数列を受け取ってそのノルムを返す関数 norm を次のように定義する．[74]

[68] Haskell では `xs = "Hello, World!"` と書く．
[69] Haskell では `zs = f <$> xs` と書く．
[70] Haskell では `zs = [f x | x <- xs]` と書く．
[71] Haskell では `z = foldl 0 (+) xs` と書く．
[72] Haskell では $\sum$ 関数を `sum` と書く．
[73] Haskell では `foldr` を用いる．
[74] Haskell では次のように書く．

$$\begin{aligned}
&\text{norm} :: [\mathbf{Double}] \to \mathbf{Double} \\
&\text{norm}\ \emptyset = 0.0 \\
&\text{norm}\ x_{\mathrm{s}} = \text{sqrt}\ \left(\sum [x * x \mid x \in x_{\mathrm{s}}]\right)
\end{aligned} \tag{73}$$

入力ファイル全体を受け取るにはアクション getContents を用いる．入力ファイルを 1 行毎のリストにするには関数 lines を用いる．各行を空白で区切ってリストに格納するには関数 words を用いる．

各文字列を数に変換するには次の関数 readDouble :: **String** → **Double** を用いる．[75]

$$\begin{aligned}
&\text{readDouble} :: \mathbf{String} \to \mathbf{Double} \\
&\text{readDouble} = \text{read}
\end{aligned} \tag{74}$$

関数 readDouble は標準関数 read に型注釈を付けたものである．

入力ファイルの各行に書かれたベクトルを対象に関数 norm を適用して，結果を書き出すには次のように書く．[76]

$$\mathsf{main} = \mathsf{print} \circ (\mathsf{norm}\ *) \circ ((\text{readDouble}\ *)\ *) \circ (\text{words}\ *) \circ \text{lines} \heartsuit \mathsf{getContents} \tag{75}$$

アクション print に代えて次の printEach を用いると，入力と出力を同じ形式にできる．[77]

$$\mathsf{printEach}\ x_{\mathrm{s}} = \mathsf{print} \mathbin{⁂} x_{\mathrm{s}} \tag{76}$$

演算子 ⁂ はアクション版のマップ演算子である．

14 関手とモナド

14.1 Maybe

計算は失敗する可能性がある．たとえば

$$z = \frac{y}{x} \tag{77}$$

のときに $x = 0$ であったとしたら，この計算は失敗する．プログラムが計算を失敗した場合，たいていのプログラマは大域ジャンプを試みる．しかし大域ジャンプは変数の書き換えを行うことであるから，別の方法が望まれる．Haskell では失敗する可能性がある場合には Maybe という機構が使える．

いま関数 f が引数 x と y を取り，$x \neq 0$ であるならば y/x を返すとする．もし $x \neq 0$ であれば失敗を意味する Nothing（ナッシング）を返すとする．すると関数 f の定義は次のようになる．

```
norm :: [haskell.Double] -> haskell.Double
norm [] = 0.0
norm xs = sqrt (sum [x * x | x <- xs])
```

[75] Haskell では次のように書く．

```
readDouble :: String -> Double
readDouble = read
```

[76] Haskell では次のように書く．

```
main = print
  . (norm <$>)
  . ((readDouble <$>) <$>)
  . (words <$>)
  . lines
  =<< getContents
```

[77] Haskell では `printEach xs = mapM print xs` と書く．

$$fxy = \texttt{if}\ x \neq 0\ \texttt{then}\ \frac{y}{x}\ \texttt{else}\ \mathrm{Nothing}\ \ldots\ \textbf{不完全な定義} \tag{78}$$

残念ながら上式は不完全である．なぜならば $x \neq 0$ のときの戻り値は数であるのに対して，$x \neq 0$ のときの戻り値は数ではないからである．そこで

$$fxy = \texttt{if}\ x \neq 0\ \texttt{then}\ \mathrm{Just}_{(y/x)}\ \texttt{else}\ \mathrm{Nothing} \tag{79}$$

とする．ここに $\mathrm{Just}_{(y/x)}$ は数 y/x から作られる，Maybe で包まれた数である．[78]

整数型 **Int** を Maybe で包む場合は $\mathbf{Maybe_{Int}}$ と書く．Maybe で包まれた型を持つ変数は $x_?$ のように小さく ? をつける．例を挙げる．[79]

$$x_? :: \mathbf{Maybe_{Int}} \tag{80}$$

Maybe で包まれた型を持つ変数は，値を持つか Nothing（ナッシング）であるかのいずれかである．値をもつ場合は $x_? = \mathrm{Just}_1$ のように書く．[80]

Maybe 変数が値を持たない場合は $x_? = \mathrm{Nothing}$ のように書く．[81]

一度 Maybe になった変数を非 Maybe に戻すことは出来ない．

14.2 Maybe に対する計算

Maybe 変数に，非 Maybe 変数を受け取る関数を適用することは出来ない．そこで特別な演算子 $\circledast$ を用いて，次のように計算する．[82]

$$z_? = f \circledast x_? \tag{81}$$

ここに関数 f は 1 引数関数で，演算子 $\circledast$ は次のように定義される．

$$\begin{aligned} \mathrm{Just}_{(fx)} &= f \circledast \mathrm{Just}_x \\ \mathrm{Nothing} &= f \circledast \mathrm{Nothing} \end{aligned} \tag{82}$$

14.3 Maybe の中のリスト

リストが Maybe の中に入っている場合は，リストの各要素に関数を適用することができる．例を挙げる．

$x_? = \mathrm{Just}_{[1,2\ldots100]}$ のとき，リストの各要素に関数 $f :: \mathbf{Int} \to \mathbf{Int}$ を適用するには次のように書く．[83]

$$z_? = (f\,*) \circledast x_? \tag{83}$$

14.4 型パラメタと型クラス

型をパラメタとして扱うことができる．任意の型を $\boldsymbol{a}$ と，ボールド体小文字で書く．ある型 $\boldsymbol{a}$ の引数を取り，同じ型を返す関数の型は次のように書ける．[84]

$$f :: \boldsymbol{a} \to \boldsymbol{a} \tag{84}$$

---

[78] Haskell では `f x y = if x /= 0 then Just y / x else Nothing` と書く．

[79] Haskell では `xm :: Maybe Int` と書く．

[80] Haskell では `xm = Just 1` と書く．

[81] Haskell では `xm = Nothing` と書く．

[82] Haskell では `zm = (+1) <$> xm` と書く．

[83] Haskell では `zm = (f <$>) <$> xm` と書く．最初の `<$>` はリストの各要素に関数 `f` を適用する演算子，2 番目の `<$>` は Maybe の中のリストの各要素に関数 `f` を適用する演算子である．

[84] Haskell では `f :: a -> a` と書く．

型パラメタには制約をつけることができる．型の集合を**型クラス**と呼び，フラクチュール体で書く．たとえば数を表す型クラスは $\mathfrak{Num}$ である．型パラメタ $\boldsymbol{a}$ が型クラス $\mathfrak{Num}$ に属するとき，上述の関数 f の型注釈は次のようになる．[85]

$$f :: \mathfrak{Num} \supset \boldsymbol{a} \Rightarrow \boldsymbol{a} \to \boldsymbol{a} \tag{85}$$

ここに $\mathfrak{Num}$ 型クラスには，整数型 **Int**，浮動小数点型 **Double** が含まれる一方，論理型 **Bool** は含まれない．

型クラスは型に制約を与える．

14.5 関手

型 $\boldsymbol{a}$ のリストの変数は $x_{\mathrm{s}} :: [\boldsymbol{a}]$ という型注釈を持つ．これは $x_{\mathrm{s}} :: []_{\boldsymbol{a}}$ のシンタックスシュガーである．[86]

型 $\boldsymbol{a}$ の Maybe の変数は $x_? :: \mathbf{Maybe}_{\boldsymbol{a}}$ という型注釈を持つ．

普段遣いの関数 $f :: \boldsymbol{a} \to \boldsymbol{a}$ をリスト変数 $x_{\mathrm{s}} :: [\boldsymbol{a}]$ に適用する場合は次のように書く．[87]

$$z_{\mathrm{s}} = f * x_{\mathrm{s}} \tag{86}$$

同じく関数 $f :: \boldsymbol{a} \to \boldsymbol{a}$ を Maybe 変数 $x_? :: \mathbf{Maybe}_{\boldsymbol{a}}$ に適用する場合は次のように書く．

$$z_? = f \circledast x_? \tag{87}$$

リストも Maybe も元の型 $\boldsymbol{a}$ から派生しており，関数適用のための特別な演算子を持つことになる．そこで，リストや Maybe は**関手**という型クラスに属する，型パラメタを伴う型であるとする．関手の型クラスを $\mathfrak{Functor}$ で表す．関手型クラスの $\boldsymbol{a}$ 型の変数を次のように型注釈する．[88]

$$x_* :: \mathfrak{Functor} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f}_{\boldsymbol{a}} \tag{88}$$

型 $[]_{\boldsymbol{a}}$ や型 $\mathbf{Maybe}_{\boldsymbol{a}}$ は型 $\boldsymbol{a}$ のところに **Int** や **Double** を代入すると具体的な型となる抽象的な型であった．今度は [] や **Maybe** のほうも $\boldsymbol{f}$ と抽象化するのである．この $\boldsymbol{f}$ は**型コンストラクタ**と呼ぶ．型コンストラクタには，具体的な引数，例えば **Int** や **Double** を与えると具体的な型になる．

型クラス $\mathfrak{Functor}$ に属する型は $\circledast$ 演算子を必ず持つ．演算子 $\circledast$ は次の形を持つ．[89]

$$z_* = f \circledast x_* \tag{89}$$

演算子 $\circledast$ の型は次のとおりである．

$$\lozenge \circledast \lozenge :: \mathfrak{Functor} \supset \boldsymbol{f} \Rightarrow (\boldsymbol{a} \to \boldsymbol{b}) \to \boldsymbol{f}_{\boldsymbol{a}} \to \boldsymbol{f}_{\boldsymbol{b}} \tag{90}$$

もし変数 x_* の型がリストであれば $\circledast = *$ であると解釈する．

14.6 関手としての関数

$$f :: \boldsymbol{r} \to \boldsymbol{q} \tag{91}$$

$$f :: \langle \boldsymbol{r} \to \blacklozenge \rangle_{\boldsymbol{q}} \tag{92}$$

---

[85] Haskell では `f :: Num a => a -> a` と書く．
[86] Haskell では `xs :: [] a` と書く．
[87] Haskell では `zs = f map xs` と書く．
[88] Haskell では `xm :: Functor f => f a` と書く．
[89] Haskell では `zm = f <$> xm` と書く．

$$f :: \langle (\rightarrow) \boldsymbol{r} \blacklozenge \rangle_{\boldsymbol{q}} \tag{93}$$

```
f :: ((->) r) q
```

$$f_2 \circ f_1 = f_2 \circledast f_1 \tag{94}$$

$$\begin{aligned} \mathrm{id} \circ f &= \mathrm{id} \circ f = f \\ (h \circ g) \circ f &= h \circ (g \circ f) \end{aligned} \tag{95}$$

15 アプリカティブ関手

15.1 アプリカティブ関手

演算子 $\circledast$ は関手型クラスの型の値に 1 引数関数を適用することを可能にした．一方で 2 引数関数を適用するのは若干面倒である．いま関数 f が 2 引数をとるとし，関手型クラスの型の変数 x_* と y_* があるとする．関数 f に変数 x_* を部分適用して関数 $f' = f \circledast x_*$ を作ると，定義によって関数 f' は関手型クラスの型の変数になる．そこで，関手型クラスの型の関数を関手型クラスの型の変数に適用する新しい演算子が必要になる．このような演算子を**アプリカティブマップ演算子**と呼び $\boxast$ で表す．アプリカティブマップ演算子を用いると 2 引数の関数適用は次のように書ける．

$$\begin{aligned} z_* &= f' \boxast y_* \\ &= f \circledast x_* \boxast y_* \end{aligned} \tag{96}$$

任意の変数または関数を関手型クラスの型に入れる**ピュア演算子**があり，次のように書く．[90]

$$z_* = (\!|x|\!) \tag{97}$$

なおピュア演算子の名称は「純粋(pure)」であるが，意味合いはむしろ「不純(impure)」のほうが近い．

ピュア演算子を用いると，式 96 は次のように書ける．[91]

$$z_* = (\!|f|\!) \boxast x_* \boxast y_* \tag{98}$$

式 98 はかつて次のように書くことが提案されたが，却下された．[92]

$$z_* = [\![f x_* y_*]\!] \ldots \textbf{採用されなかった文法} \tag{99}$$

ピュア演算子とアプリカティブマップ演算子を必ず持つ関手のことを**アプリカティブ関手**と呼び $\mathfrak{Applicative}$ で表す．

いま関数 $f :: \boldsymbol{a} \rightarrow \boldsymbol{b}$ に対して，新たな関数 f_* ただし $f_* = (\!|f|\!)$ を作ったとすると，関数 f_* は次の型を持つ．

$$f_* :: \mathfrak{Applicative} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f}_{\boldsymbol{a} \rightarrow \boldsymbol{b}} \tag{100}$$

アプリカティブマップ演算子は変数 $x_* :: \mathfrak{Applicative} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f}_{\boldsymbol{a}}$ に対して，関数 f_* を $z_* = f_* \boxast x_*$ のように作用させる．変数 z_* の型は $z_* :: \mathfrak{Applicative} \supset \boldsymbol{f} \Rightarrow \boldsymbol{f}_{\boldsymbol{b}}$ である．

[90] Haskell では `z = pure x` と書く．
[91] Haskell では `zm = (pure f) <*> xm <*> ym` と書く．
[92] 現在の Haskell では `z = liftA2 f x y` と書くことで代用されている．元の提案は `z = [|f x y|]` であった．

15.2 モナド

15.3 種

15.4 Data

15.5 型クラスとインスタンス

15.6 IO モナド

15.7 Do 構文

❇️ **TK**

```
z = do { x' <- x; y' <- y; f x'; g y' }
```

15.8 モノイド則

❇️ **TK**

$(\mathbb{Z}, +, 0)$ はモノイドである．

同じ型を持つ関数を集めた関数の集合 $\mathbb{F}$ があるとする．任意の関数 $f \in \mathbb{F}$ について $\mathrm{id}\ f = f$ なる関数 id があり，かつ任意の関数 $f, g, h \in \mathbb{F}$ に対して $(h \circ g) \circ f) = h \circ (g \circ f)$ が成り立つとする．このとき組み合わせ $(\mathbb{F}, \circ, \mathrm{id})$ は**モノイド**であるという．

- 単位元の存在
- 結合律

15.9 関手則

❇️ **TK**

$$\begin{aligned} \mathrm{id} \circledast x_* &= \mathrm{id}\ x_* \\ (g \circ f) \circledast x_* &= ((g \circledast) \circ (f \circledast))x_* \end{aligned} \tag{101}$$

15.10 アプリカティブ関手則

❇️ **TK**

$$\begin{aligned} (\mathrm{id}) \boxast x_* &= x_* \\ (f) \boxast (x) &= (\diamond\ \$\ x) \boxast (f) \\ &= (fx) \\ (\diamond \circ \diamond) \boxast h_* \boxast g_* \boxast f_* &= h_* \boxast (g_* \boxast f_*) \end{aligned} \tag{102}$$

15.11 モナド則

❇️ **TK**

$$\begin{aligned} \mu \heartsuit (\!|x|\!) &= \mu x \\ (\!|\diamond|\!) \heartsuit x_* &= x_* \\ \nu \heartsuit \mu \heartsuit x_* &= \nu \heartsuit (\mu \heartsuit x_*) \end{aligned} \tag{103}$$

$(\mathbb{M}, \heartsuit, (\!|\diamond|\!))$ はモノイドである．

関数 μ に作用するクライスリスター演算子 $\star$ を $\mu^\star = (\mu \heartsuit \diamond)$ と定義する．クライスリスターを用いると，モナド則は次のように書き直せる．

$$\begin{aligned} \mu^\star(x) &= \mu x \\ (\diamond)^\star x_* &= x_* \\ (\nu^\star \mu)^\star x_* &= \nu^\star(\mu^\star x_*) \end{aligned} \tag{104}$$

15.12 クラスの定義

16 Test Part

16.1 Function application

$$z = fx \tag{105}$$

```
z = f x
```

16.2 List map

$$z_{\mathrm{s}} = f * x_{\mathrm{s}} \tag{106}$$

```
zs = map f xs
```

or

```
zs = f `map` xs
```

16.3 Maybe map

$$z_{?} = f \circledast x_{?} \tag{107}$$

```
zm =  fmap f xm
```

or

```
zm =  f <$> xm
```

16.4 Functor map

$$z_{*} = f \circledast x_{*} \tag{108}$$

```
zm = f <$> xm
```

16.5 Applicative map

$$z_{*} = f_{*} \boxast x_{*} \tag{109}$$

```
zm = fm <*> xm
```

16.6 Apllicative map 2

$$z_{*} = g_{*} \boxast x_{*} \boxast y_{*} \tag{110}$$

```
zm = gm <*> xm <*> ym
```

Or,

$$z_{*} = g \circledast x_{*} \boxast y_{*} \tag{111}$$

```
zm = g <$> xm <*> ym
```

16.7 Monadic function application

$$z_* = fx \tag{112}$$

```
zm = f x
```

16.8 Bind

$$z_* = f \heartsuit x_* \tag{113}$$

```
zm = f =<< xm
```

16.9 Double bind

$$z_* = g \heartsuit f \heartsuit x_* \tag{114}$$

```
zm = g =<< f =<< xm
```

16.10 Contextual map

$$\mathsf{printEach} = \mathsf{print} \mathbin{⁂} x_{\mathrm{s}} \tag{115}$$

```
printEach = print `mapM` xs
```

17 Test Part 2

17.1 Lambda

$$(\backslash\, x \mapsto x + 1)y \tag{116}$$

```
(\x -> x + 1) y
```

17.2 Operators

Scalar:

$$z = fx \tag{117}$$

```
z = f x
```

List:

$$z_{\mathrm{s}} = f * x_{\mathrm{s}} \tag{118}$$

```
zs = f `map` xs
```

Maybe:

$$z_{\mathrm{m}} = f \circledast x_{\mathrm{m}} \tag{119}$$

```
zm = f <$> xm
```

Any functor:

$$z_* = f \circledast x_* \tag{120}$$

```
z = f <$> x
```

Applicative version:

$$z_* = (f) \boxast x_* \tag{121}$$

```
z = f <*> x
```

Composition.

$$h = g \circ f \tag{122}$$

```
h = f . g
```

Scalar to contextual:

$$z_* = \tilde{f} x \tag{123}$$

```
z = f x
```

Monadic composition.

$$\tilde{h} = \tilde{g} \heartsuit \tilde{f} \tag{124}$$

```
h = g =<< f
```

17.3 Types

Scaler type.

$$x :: \mathbf{Int} \tag{125}$$

Function type.

$$f :: \boldsymbol{a} \to \boldsymbol{b} \tag{126}$$

Functor type.

$$\begin{aligned} x_{\mathrm{s}} &:: [\mathbf{Int}] \\ x_* &:: \mathfrak{Monad} \supset \boldsymbol{m}, \mathfrak{Eq} \supset \boldsymbol{a} \Rightarrow \boldsymbol{m}_{\boldsymbol{a}} \\ f &:: (\boldsymbol{r} \to \blacklozenge)_{\boldsymbol{q}} \end{aligned} \tag{127}$$

型引数は添字として表現することにした.